



Integração & Implantação

Passo a Passo do Zero

Assistência Técnica de Drones DJI Agras

Plataforma multiempresa · OS por estágios · orçamento · IA preditiva · auditoria

geoAG — Goiânia-GO

Av. Francisco de Melo, Quadra 41, Lote 06 — 74.345-210, Vila Rosa
contato@geoag.com.br · +55 (62) 3914-4516 · <https://geoag.com.br>

Documento de integração do cliente · Julho de 2026

0. Visão geral — o que é e o que a integração entregou

A plataforma é um **SaaS de assistência técnica de drones DJI Agras**: abertura de chamados, Ordem de Serviço por estágios com aprovação, orçamento (PDF + WhatsApp), estoque de peças com custo/preço, IA preditiva, financeiro, auditoria imutável e rastreabilidade vitalícia por Serial Number.

A **integração real do geoAG** (white-label + onboarding) entregou:

#	Entrega	Onde
1	Marca geoAG: nome, cores (verde/laranja), logo, favicon, título da aba	brand.ts, styles.css, index.html
2	Textos do app com a marca (topo, login, WhatsApp, rodapé)	App.tsx, i18n.ts
3	Contato real do geoAG no registro da empresa e no rodapé dos PDFs	Empresa + V14, BrandProperties
4	Onboarding de produção: cria a empresa geoAG + admin no 1º boot	GeoagOnboardingSeeder.java
5	Configuração de produção (env, Docker, CORS)	.env.example, docker-compose.yml

Trocar de cliente no futuro = editar dois pontos: **frontend/src/brand.ts** e as chaves **app.marca.*** do backend (por variáveis de ambiente). Validação: 41 testes de integração verdes e frontend compilando sem erros.

Etapa 1 — Pré-requisitos

Escolha **um** caminho de instalação:

- **Caminho A — Docker (recomendado p/ produção)**: servidor Linux (ou Windows com Docker Desktop) com Docker + Docker Compose. Sobe PostgreSQL + backend + frontend juntos.
- **Caminho B — Windows 2 cliques (treinamento)**: Windows 10/11 com winget. Os .bat instalam Java 21 e Node sozinhos.
- **Caminho C — Manual (dev)**: Java 21, Node 18+, PostgreSQL 14+.

Para produção com domínio: um **domínio** (ex.: app.geoag.com.br) apontando para o servidor e as portas 80/443 liberadas.

Etapa 2 — Obter o projeto

Baixe o projeto no servidor (ou copie a pasta, ex.: /opt/geoag):

```
git clone <URL-DO-REPOSITÓRIO> geoag
cd geoag
```

Etapa 3 — Configurar os segredos (.env)

Copie o modelo e edite os valores:

```
cp .env.example .env
```

Ajuste no `.env` (todos importam):

Variável	O que colocar
DB_PASSWORD	uma senha forte para o banco
KAIROS_JWT_SECRET	um segredo longo e único (32+ caracteres) — gere abaixo
KAIROS_CORS_ORIGINS	o domínio do frontend, ex.: <code>https://app.geoag.com.br</code>
GEOAG_ONBOARDING_SEED	true no primeiro boot (cria a empresa geoAG)
GEOAG_CNPJ	o CNPJ real do geoAG (troque o placeholder)
GEOAG_ADMIN_EMAIL / _SENHA	acesso inicial do admin — troque a senha no 1º login
KAIROS_DEMO_SEED	false em produção

Gerar um KAIROS_JWT_SECRET forte:

```
openssl rand -base64 48
```

Nunca versiona o arquivo `.env` — ele já está no `.gitignore`.

Etapa 4 — Subir a stack

Caminho A — Docker (produção)

```
docker compose up --build -d
```

Sobe três serviços; o Flyway cria as tabelas sozinho (migrações V1...V14):

- **Aplicação (web):** `http://localhost:8080` (ou o IP do servidor)
- **API / health:** `http://localhost:8081/api/v1/health`
- **PostgreSQL:** porta 5433 (volume persistente geoag-db)

Caminho B — Windows 2 cliques (treinamento)

- Dois cliques em `install.bat` (instala Java/Node e compila).
- Dois cliques em `iniciar.bat` (sobe tudo e abre o navegador).

Esse modo usa dados de demonstração em memória — ótimo para treinar a equipe; não é o ambiente de produção.

Caminho C — Manual (dev)

```
cd backend && ./mvnw spring-boot:run
cd frontend && npm install && npm run dev
```

Etapa 5 — Onboarding automático do geoAG (1º boot)

Com `GEOAG_ONBOARDING_SEED=true`, no primeiro boot o sistema cria sozinho a **empresa geoAG** (CNPJ + contato) e o **usuário ADMIN**. Confirme no log do backend:

```
[geoag] Pronto. Empresa 'geoAG' criada (tenant ...). ADMIN: contato@geoag.com.br
- TROQUE A SENHA no primeiro acesso.
```

É **idempotente**: se a empresa já existe (mesmo CNPJ), não duplica.

Primeiro acesso:

- Abra a aplicação (ex.: `http://localhost:8080`).
- Entre com `GEOAG_ADMIN_EMAIL` e `GEOAG_ADMIN_SENHA`.
- Vá em Usuários e **troque a senha** do administrador.
- (Opcional) volte `GEOAG_ONBOARDING_SEED=false` no `.env`.

Etapa 6 — Configurar a operação (como ADMIN)

- **Usuários** — cadastre a equipe: TÉCNICO (executa as OS); o CLIENTE é criado automaticamente ao abrir um chamado.
- **Estoque (KSI)** — cadastre as peças com custo de compra, preço de venda e fornecedor (habilita margem, giro e reposição automática).
- **Aeronaves** — registre os drones por Serial Number (rastreabilidade).
- **Chamados** — abra o primeiro chamado; o sistema gera protocolo, cria o acesso do cliente e envia o login por WhatsApp.

Fluxo da OS: FILA -> ORÇAMENTO -> AGUARDANDO APROVAÇÃO -> APROVADO -> ESTÁGIO 1 -> (Estágio 2 se preciso) -> FINALIZADO (baixa de peças + histórico + auditoria).

Etapa 7 — Domínio e HTTPS (produção)

Aponte `app.geoag.com.br` para o servidor e use um reverse proxy com HTTPS automático. Exemplo com **Caddy**:

```
app.geoag.com.br {
    reverse_proxy localhost:8080
}
```

Depois ajuste `KAIROS_CORS_ORIGINS=https://app.geoag.com.br` e reinicie (docker `compose up -d`).

Etapa 8 — Checklist de validação

Antes de liberar para a equipe, confirme:

- [] `docker compose up --build -d` sem erros; `/api/v1/health` responde.
- [] Login do ADMIN funciona; senha inicial trocada.
- [] Marca geoAG no topo, no título da aba e no rodapé (verde/laranja).
- [] Abrir um chamado gera protocolo; o botão WhatsApp cita 'geoAG'.
- [] PDF de Auditoria traz geoAG + contato no cabeçalho.
- [] `GEOAG_CNPJ` real; `KAIROS_JWT_SECRET` e `DB_PASSWORD` fortes.
- [] `KAIROS_DEMO_SEED=false`; HTTPS ativo; CORS com o domínio.
- [] Backup do banco agendado (Etapa 9).

Etapa 9 — Backup, restauração e atualização

Backup diário (agende no cron):

```
docker compose exec postgres pg_dump -U geoag geoag > backup-$(date +%F).sql
```

Restauração:

```
docker compose exec -T postgres psql -U geoag geoag < backup-2026-01-01.sql
```

Atualizar a versão: substitua os arquivos e rode `docker compose up --build -d` — o Flyway aplica as novas migrações sozinho.

Etapa 10 — Onde a marca vive (referência técnica)

Camada	Arquivo / chave	Papel
Frontend	frontend/src/brand.ts	Fonte única: nome, cores, contato, WhatsApp
Frontend	styles.css (:root)	Paleta verde/laranja
Frontend	index.html	Título da aba + favicon
Frontend	i18n.ts, App.tsx	Textos com a marca
Backend	app.marca.* (APP_MARCA_*)	Marca nos relatórios PDF
Backend	BrandProperties.java	Bind das propriedades de marca
Backend	GeoagOnboardingSeeder.java	Onboarding da empresa geoAG
Banco	V14__empresa_contato.sql	Contato no registro da empresa

Etapa 11 — Solução de problemas rápida

Sintoma	Solução
Onboarding não criou a empresa	confirme <code>GEOAG_ONBOARDING_SEED=true</code> e veja o log [geoag]
'Já existe empresa com o documento'	o CNPJ já foi usado — o seed é idempotente, ignore
Frontend não fala com a API	inclua o domínio em <code>KAIROS_CORS_ORIGINS</code>
PDF de auditoria sem a marca	defina <code>APP_MARCA_*</code> (ou use os defaults geoAG)
Login bloqueado	ADMIN -> Usuários -> reativar

Suporte geoAG · contato@geoag.com.br · +55 (62) 3914-4516 · <https://geoag.com.br>